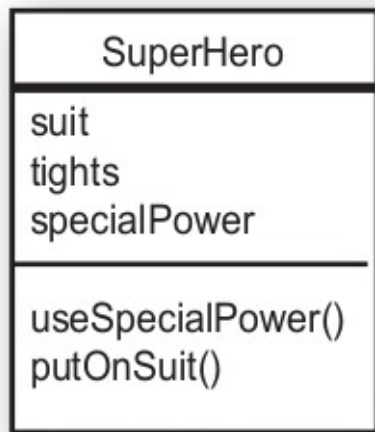


Better Living in Objectville

We were underpaid, overworked coders 'till we tried the Polymorphism Plan. But thanks to the Plan, our future is bright. Yours can be too!



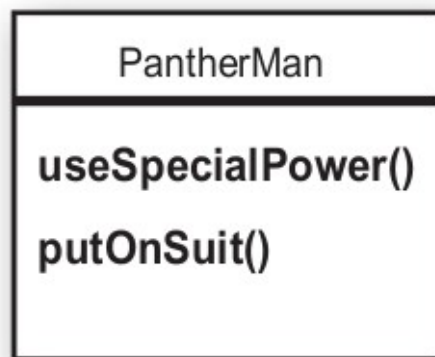
**superclass
(more abstract)**



**instance variables
(state, attributes)**

**methods
(behavior)**

**subclasses
(more specific)**



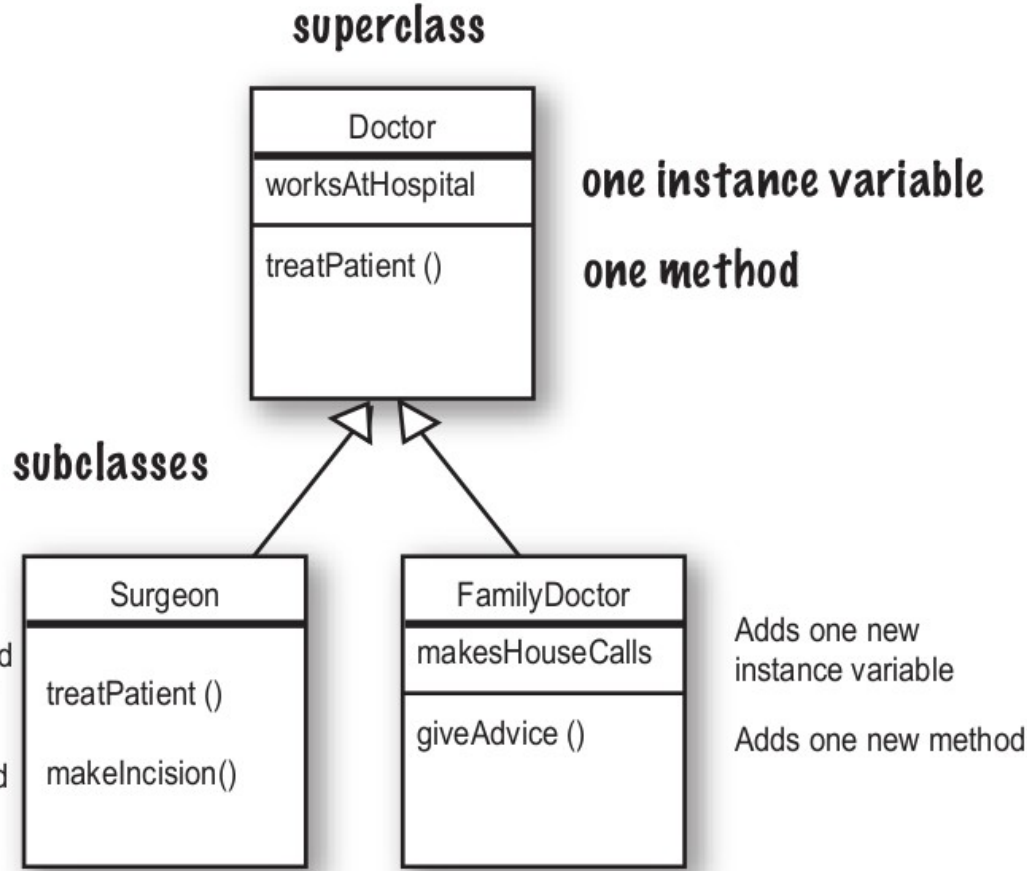
**Overriding
methods**



An inheritance example:

```
public class Doctor {  
  
    boolean worksAtHospital;  
  
    void treatPatient() {  
        // perform a checkup  
    }  
}  
  
public class FamilyDoctor extends Doctor {  
  
    boolean makesHouseCalls;  
  
    void giveAdvice() {  
        // give homespun advice  
    }  
}  
  
public class Surgeon extends Doctor {  
  
    void treatPatient() {  
        // perform surgery  
    }  
  
    void makeIncision() {  
        // make incision (yikes!)  
    }  
}
```





How many instance variables does Surgeon have? _____

How many instance variables does FamilyDoctor have? _____

How many methods does Doctor have? _____

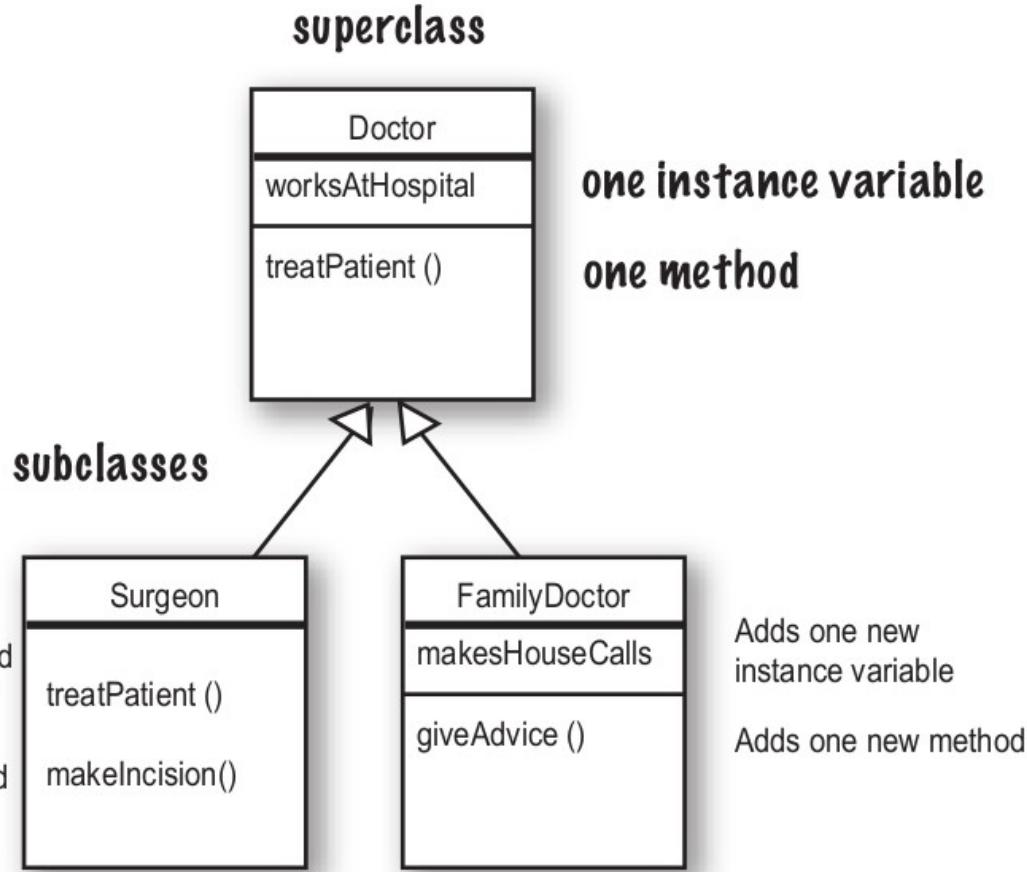
How many methods does Surgeon have? _____

How many methods does FamilyDoctor have? _____

Can a FamilyDoctor do treatPatient()? _____

Can a FamilyDoctor do makeIncision()? _____

—————> **Yours to solve.**



How many instance variables does Surgeon have? 1

How many instance variables does FamilyDoctor have? 2

How many methods does Doctor have? 1

How many methods does Surgeon have? 2

How many methods does FamilyDoctor have? 2

Can a FamilyDoctor do treatPatient()? Y

Can a FamilyDoctor do makeIncision()? N

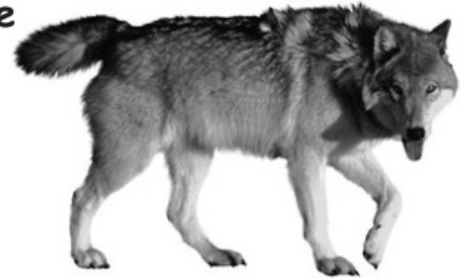
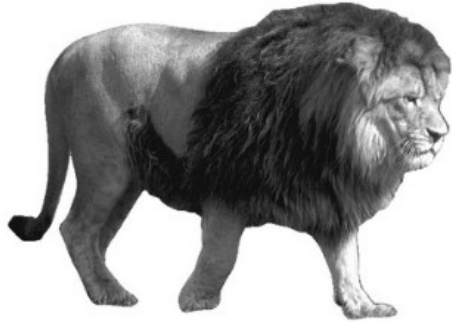
→ **Yours to solve.**

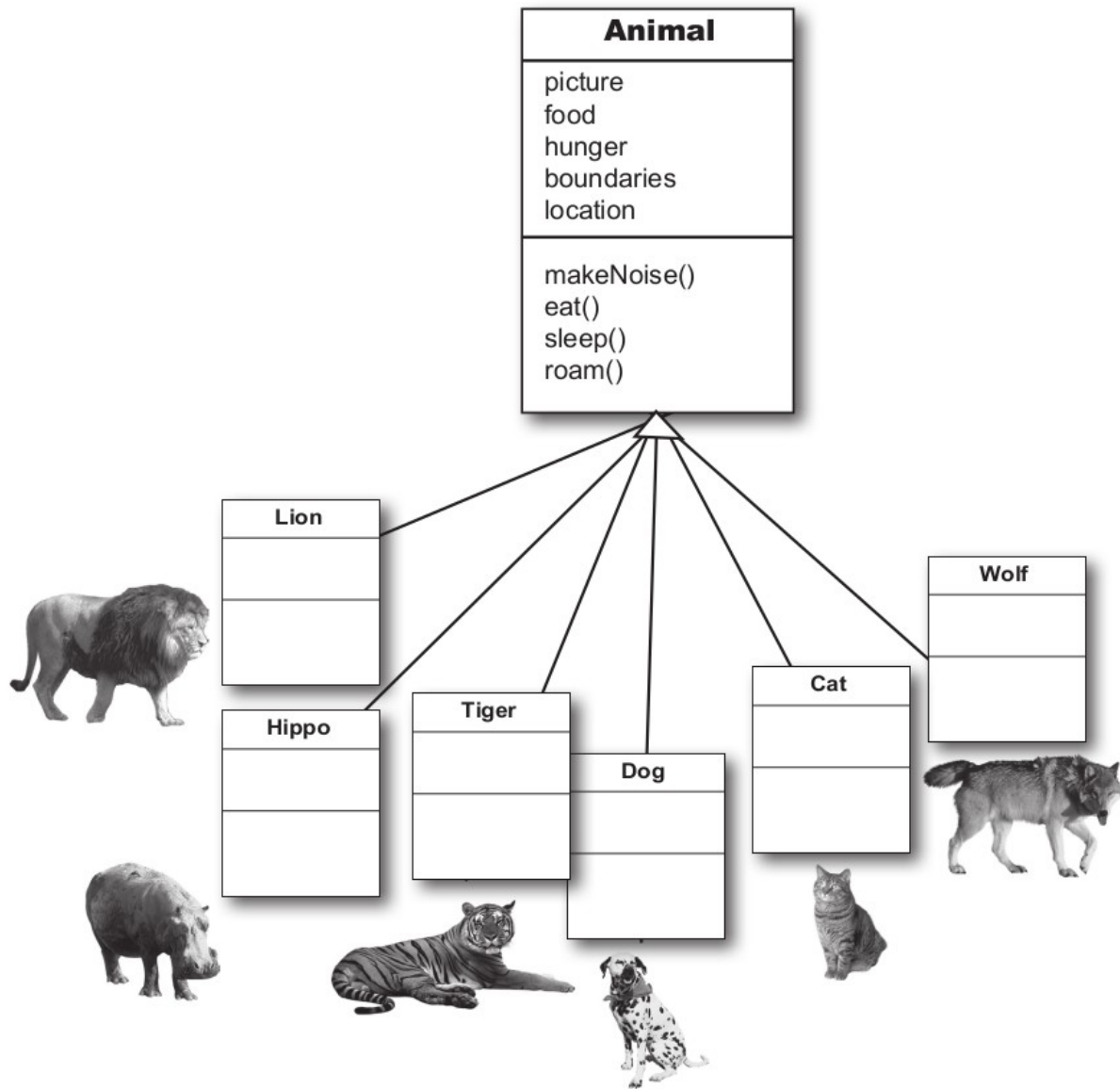
1

Look for objects that have common attributes and behaviors.

What do these six types have in common? This helps you to abstract out behaviors. (step 2)

How are these types related? This helps you to define the inheritance tree relationships (steps 4-5)



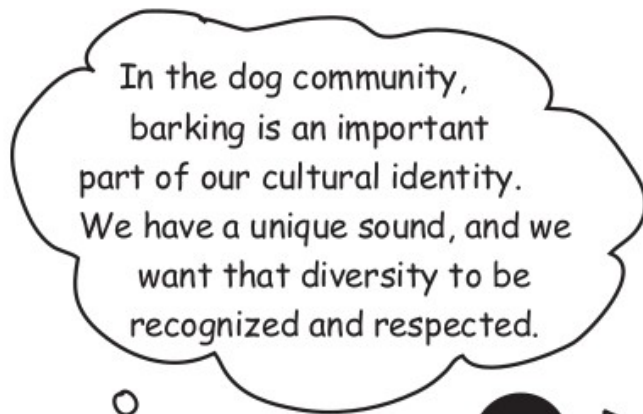


2

Design a class that represents the common state and behavior.

These objects are all animals, so we'll make a common superclass called *Animal*.

We'll put in methods and instance variables that all animals might need.



3

Decide if a subclass needs behaviors (method implementations) that are specific to that particular subclass type.

Looking at the **Animal** class, we decide that **eat()** and **makeNoise()** should be overridden by the individual subclasses.



We better override these two methods, **eat()** and **makeNoise()**, so that each animal type can define its own specific behavior for eating and making noise. For now, it looks like **sleep()** and **roam()** can stay generic.

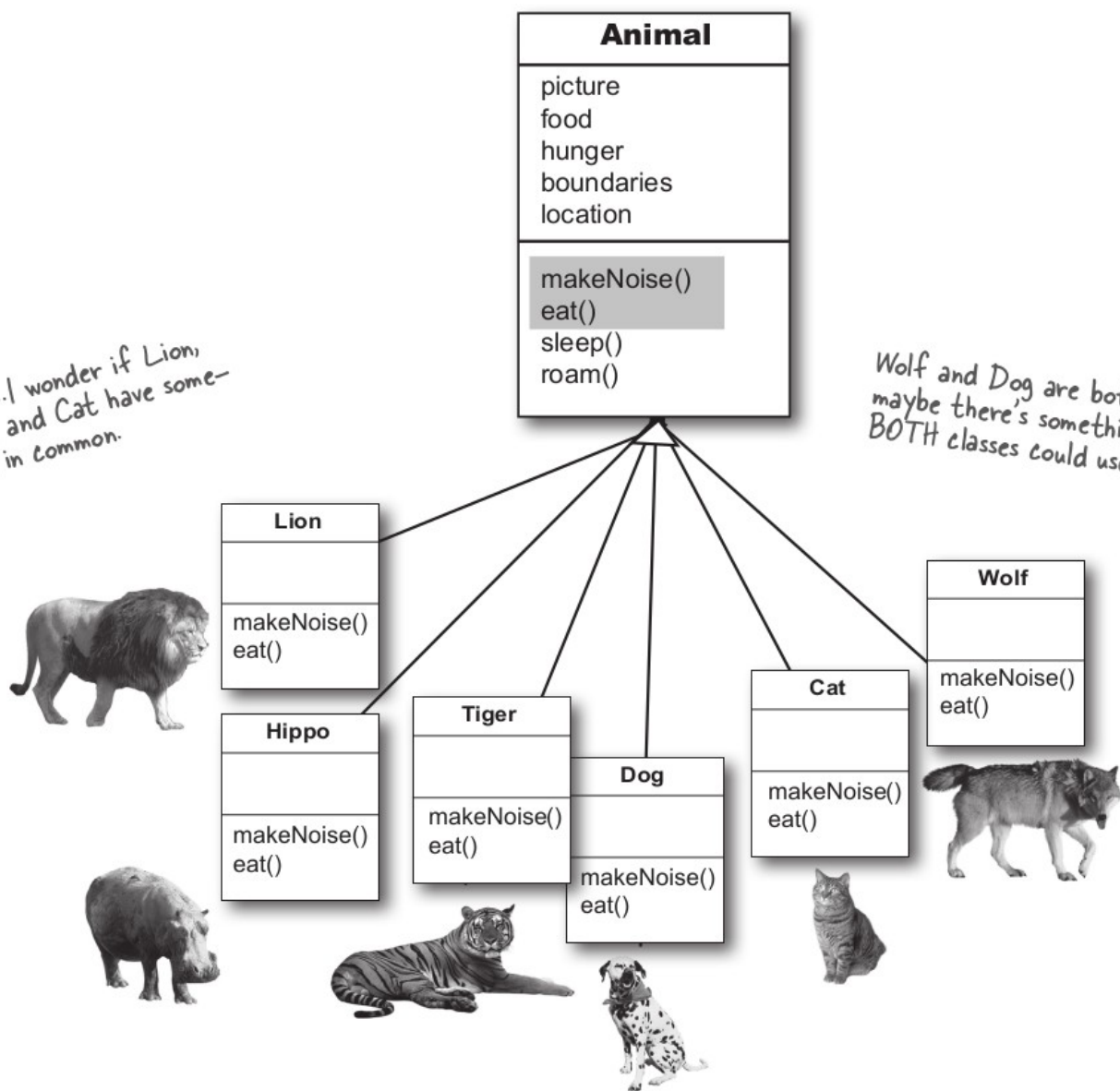
Animal
picture food hunger boundaries location
makeNoise() eat() sleep() roam()

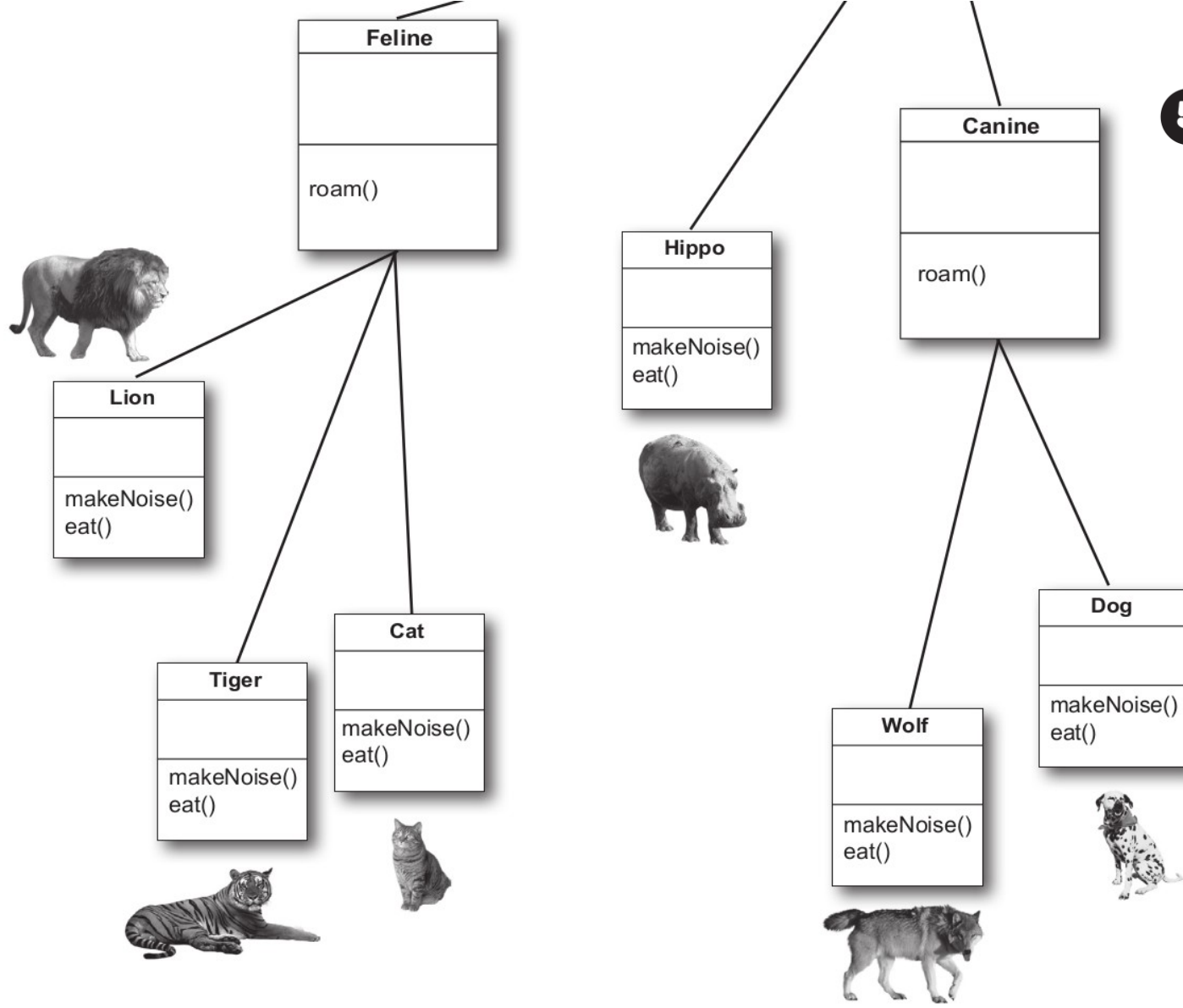
Look for more opportunities to use abstraction, by finding two or more *subclasses* that might need common behavior.

We look at our classes and see that Wolf and Dog might have some behavior in common, and the same goes for Lion, Tiger, and Cat.

Hmmm...I wonder if Lion, Tiger, and Cat have something in common.

Wolf and Dog are both canines... maybe there's something that BOTH classes could use...





5

Finish the class hierarchy

Since animals already have an organizational hierarchy (the whole kingdom, genus, phylum thing), we can use the level that makes the most sense for class design. We'll use the biological "families" to organize the animals by making a Feline class and a Canine class.

We decide that Canines could use a common `roam()` method, because they tend to move in packs. We also see that Felines could use a common `roam()` method, because they tend to avoid others of their own kind. We'll let Hippo continue to use its inherited `roam()` method—the generic one it gets from `Animal`.

So we're done with the design for now; we'll come back to it later in the chapter.

Make a new Wolf object

```
Wolf w = new Wolf();
```

Calls the version in Wolf

```
w.makeNoise();
```

Calls the version in Canine

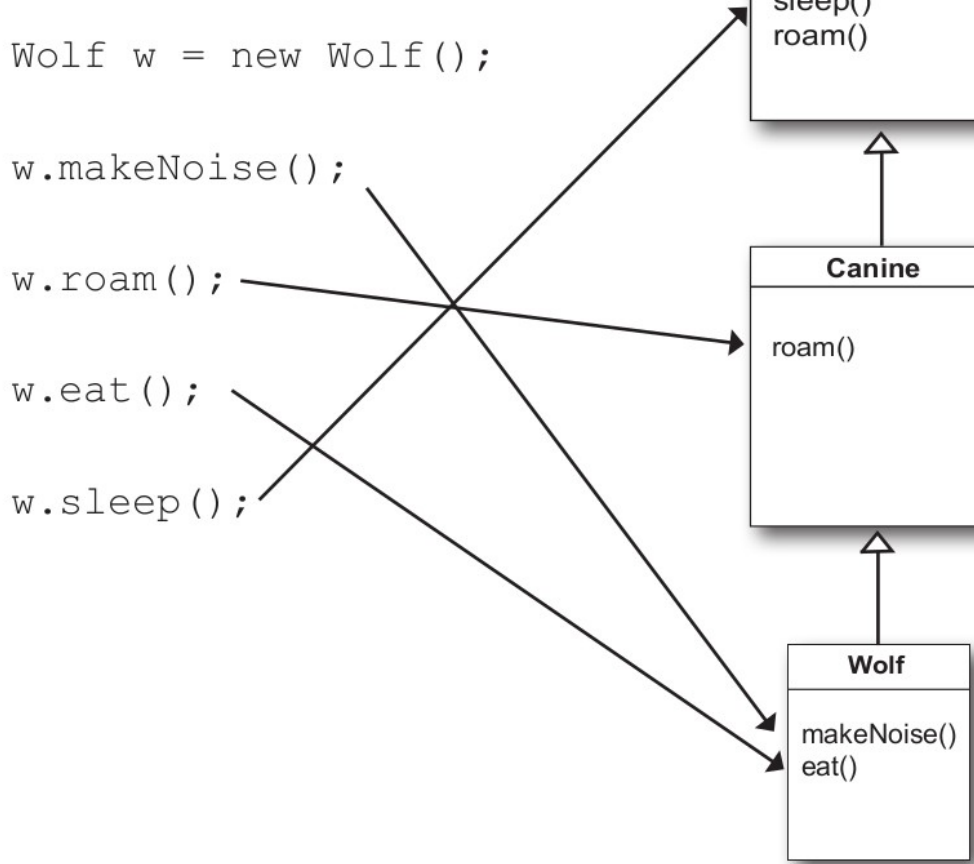
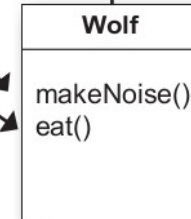
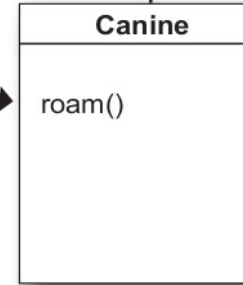
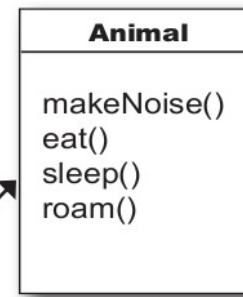
```
w.roam();
```

Calls the version in Wolf

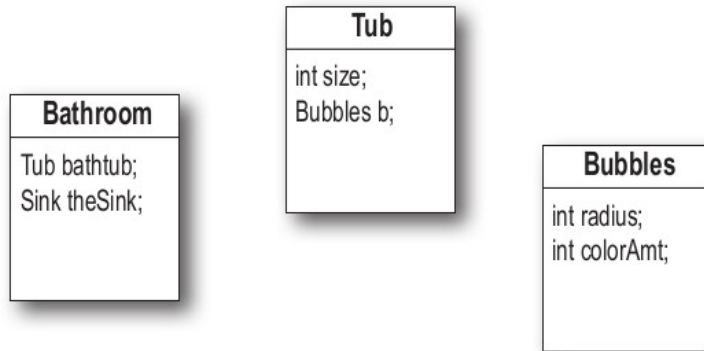
```
w.eat();
```

Calls the version in Animal

```
w.sleep();
```

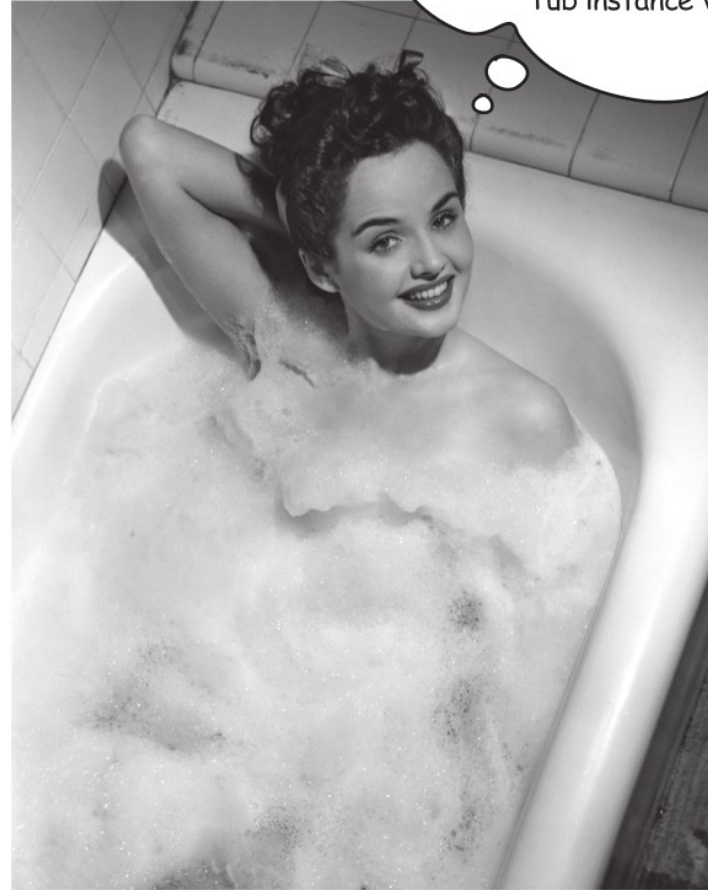


Using IS-A and HAS-A



Bathroom HAS-A Tub and Tub HAS-A Bubbles.
But nobody inherits from (extends) anybody else.

Does it make sense to say a
Tub IS-A Bathroom? Or a Bathroom
IS-A Tub? Well, it doesn't to me. The
relationship between my Tub and my
Bathroom is HAS-A. Bathroom HAS-A
Tub. That means Bathroom has a
Tub instance variable.



Make it Stick

Java is
Pass
by value

threads
wait()
notify()

Wash
Cat

Roses are red, violets are blue.

Square is-a **Shape**, the reverse isn't true.

Roses are red, violets are dear.

Beer is-a **Drink**, but not all **drinks** are **beer**.

OK, your turn. Make one that shows the one-way-ness of the IS-A relationship. Remember, if X **extends** Y, X **IS-A** Y must make sense.



Sharpen your pencil

Put a check next to the relationships that make sense.

- ☐ Oven extends Kitchen
- ☐ Guitar extends Instrument
- ☐ Person extends Employee
- ☐ Ferrari extends Engine
- ☐ FriedEgg extends Food
- ☐ Beagle extends Pet
- ☐ Container extends Jar
- ☐ Metal extends Titanium
- ☐ GratefulDead extends Band
- ☐ Blonde extends Smart
- ☐ Beverage extends Martini



Sharpen your pencil

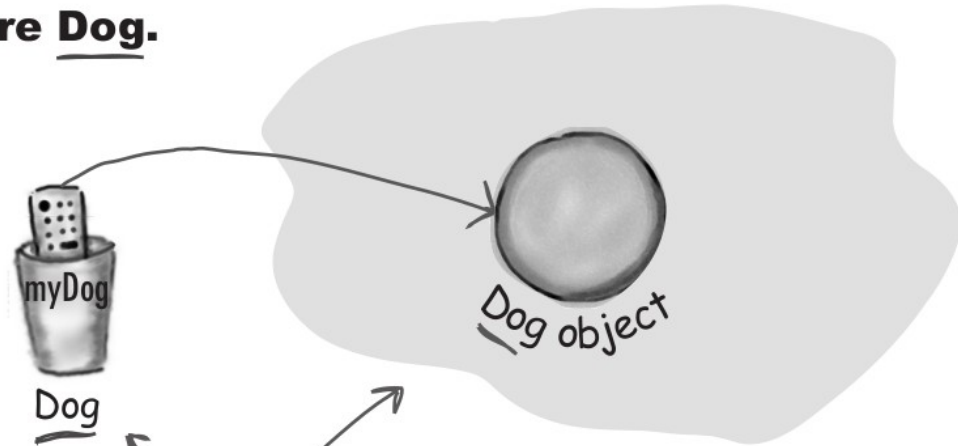
Put a check next to the relationships that make sense.

- ☐ Oven extends Kitchen
- ☒ Guitar extends Instrument
- ☐ Person extends Employee
- ☐ Ferrari extends Engine
- ☒ FriedEgg extends Food
- ☒ Beagle extends Pet
- ☐ Container extends Jar
- ☐ Metal extends Titanium
- ☒ GratefulDead extends Band
- ☒ Blonde extends Smart
- ☐ Beverage extends Martini

```
1 Dog myDog 3 = 2 new Dog ();
```

The important point is that the reference type AND the object type are the same.

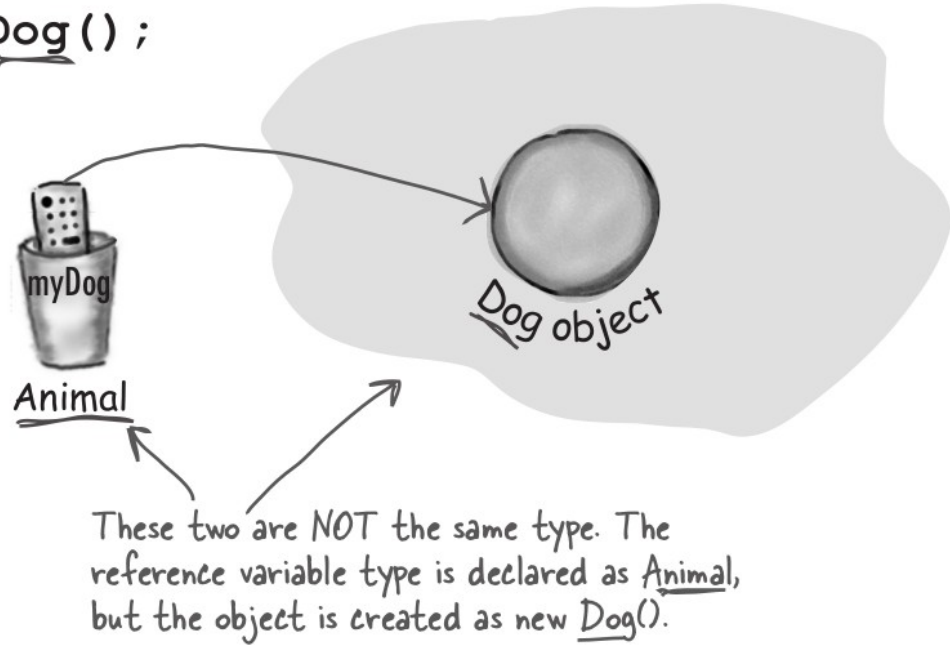
In this example, both are Dog.



These two are the same type. The reference variable type is declared as Dog, and the object is created as new Dog().

But with polymorphism, the reference type and the object type can be *different*.

```
Animal myDog = new Dog();
```



Declare an array of type Animal. In other words,
an array that will hold objects of type Animal.

```
Animal[] animals = new Animal[5];
```

```
animals[0] = new Dog();
```

```
animals[1] = new Cat();
```

```
animals[2] = new Wolf();
```

```
animals[3] = new Hippo();
```

```
animals[4] = new Lion();
```

But look what you get to do...you can put ANY subclass
of Animal in the Animal array!

```
for (Animal animal : animals) {
```

And here's the best polymorphic part (the
raison d'être for the whole example): you
get to loop through the array and call one
of the Animal-class methods, and every
object does the right thing!

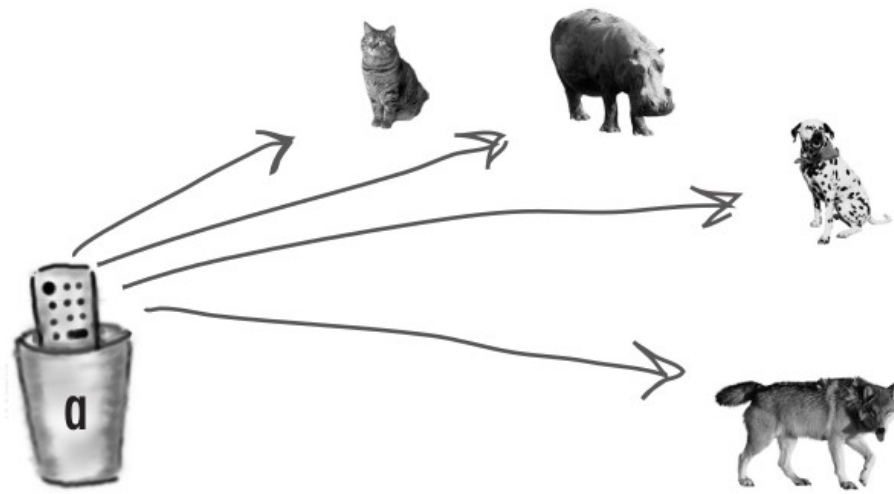
```
    animal.eat();
```

```
    animal.roam();
```

```
}
```

On the first pass through the loop, 'animal' is a Dog,
so you get the Dog's eat() method. On the next pass,
'animal' is a Cat, so you get the Cat's eat() method.

Same with roam().



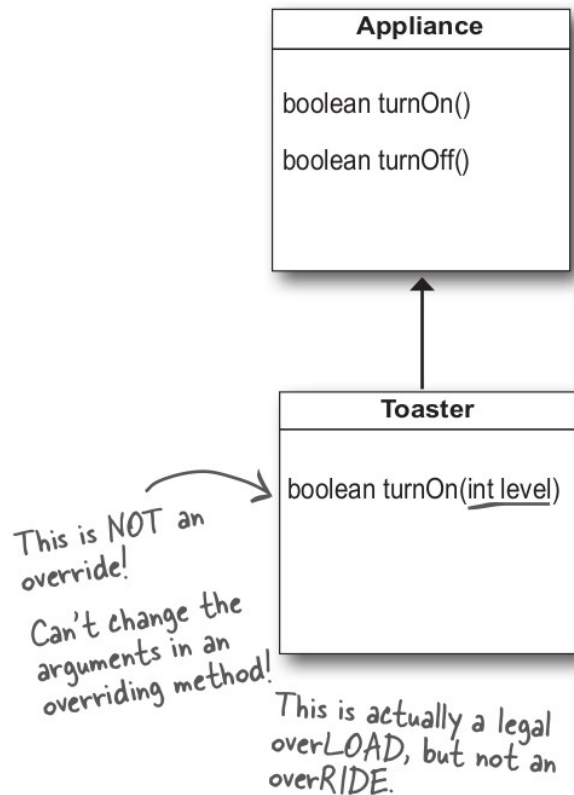
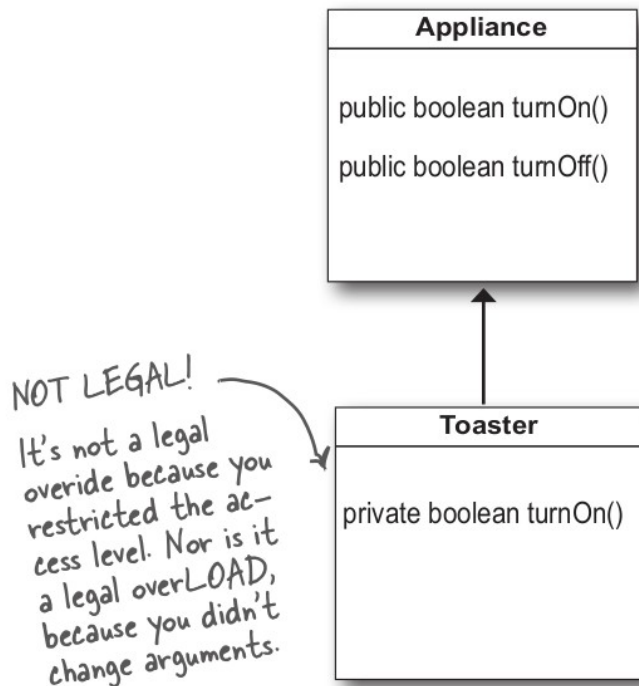
```
class Vet {  
    public void giveShot(Animal a) {  
        // do horrible things to the Animal at  
        // the other end of the 'a' parameter  
        a.makeNoise();  
    }  
}
```

The 'a' parameter can take ANY Animal type as the argument. And when the Vet is done giving the shot, it tells the Animal to makeNoise(), and whatever Animal is really out there on the heap, that's whose makeNoise() method will run.

With polymorphism, you can write code that doesn't have to change when you introduce new subclass types into the program.

Keeping the contract: rules for overriding

- ① **Arguments must be the same, and return types must be compatible.**
- ② **The method can't be less accessible.**



The methods *are* the contract

Overloading a method

An overloaded method is just a different method that happens to have the same method name. It has nothing to do with inheritance and polymorphism. An overloaded method is NOT the same as an overridden method.

① **The return types can be different.**

You're free to change the return types in overloaded methods, as long as the argument lists are different.

② **You can't change ONLY the return type.**

If only the return type is different, it's not a valid *overload*—the compiler will assume you're trying to *override* the method. And even *that* won't be legal unless the return type is a subtype of the return type declared in the superclass. To overload a method, you **MUST** change the argument list, although you *can* change the return type to anything.

③ **You can vary the access levels in any direction.**

You're free to overload a method with a method that's more restrictive. It doesn't matter, since the new method isn't obligated to fulfill the contract of the overloaded method.

Legal examples of method overloading:

```
public class Overloads {
    String uniqueID;

    public int addNums(int a, int b) {
        return a + b;
    }

    public double addNums(double a, double b) {
        return a + b;
    }

    public void setUniqueID(String theID) {
        // lots of validation code, and then:
        uniqueID = theID;
    }

    public void setUniqueID(int ssNumber) {
        String numString = "" + ssNumber;
        setUniqueID(numString);
    }
}
```

```

public class MonsterTestDrive {
    public static void main(String [] args) {
        Monster [] ma = new Monster[3];
        ma[0] = new Vampire();
        ma[1] = new Dragon();
        ma[2] = new Monster();
        for(int x = 0; x < 3; x++) {
            ma[x].frighten(x);
        }
    }
}

```



A

```

class Monster {
}

```

B

```

class Vampire extends Monster {
}

```

```

class Dragon extends Monster {
    boolean frighten(int degree) {
        System.out.println("breath fire");
        return true;
    }
}

```

File Edit Window Help SaveYourself

0 - iams MonsterTestDrive

1

A

```

boolean frighten(int d) {
    System.out.println("arrrrgh");
    return true;
}

```

B

```

boolean frighten(int x) {
    System.out.println("a bite?");
    return false;
}

```

2

A

```

boolean frighten(int x) {
    System.out.println("arrrrgh");
    return true;
}

```

B

```

int frighten(int f) {
    System.out.println("a bite?");
    return 1;
}

```

3

A

```

boolean frighten(int x) {
    System.out.println("arrrrgh");
    return false;
}

```

B

```

boolean scare(int x) {
    System.out.println("a bite?");
    return true;
}

```

```
public class MonsterTestDrive {
    public static void main(String [] args) {
        Monster [] ma = new Monster[3];
        ma[0] = new Vampire();
        ma[1] = new Dragon();
        ma[2] = new Monster();
        for(int x = 0; x < 3; x++) {
            ma[x].frighten(x);
        }
    }
}
```



A

```
class Monster {
}
```

B

```
class Vampire extends Monster {
}
```

```
class Dragon extends Monster {
    boolean frighten(int degree) {
        System.out.println("breath fire");
        return true;
    }
}
```

File Edit Window Help SaveYourself

% java MonsterTestDrive

1

```
boolean frighten(int d) {
    System.out.println("arrrrgh");
    return true;
}
```

A

```
boolean frighten(int x) {
    System.out.println("a bite?");
    return false;
}
```

2

```
boolean frighten(int x) {
    System.out.println("arrrrgh");
    return true;
}
```

A

```
int frighten(int f) {
    System.out.println("a bite?");
    return 1;
}
```

B

```
boolean frighten(int x) {
    System.out.println("arrrrgh");
    return false;
}
```

3

```
boolean frighten(int x) {
    System.out.println("arrrrgh");
    return false;
}
```

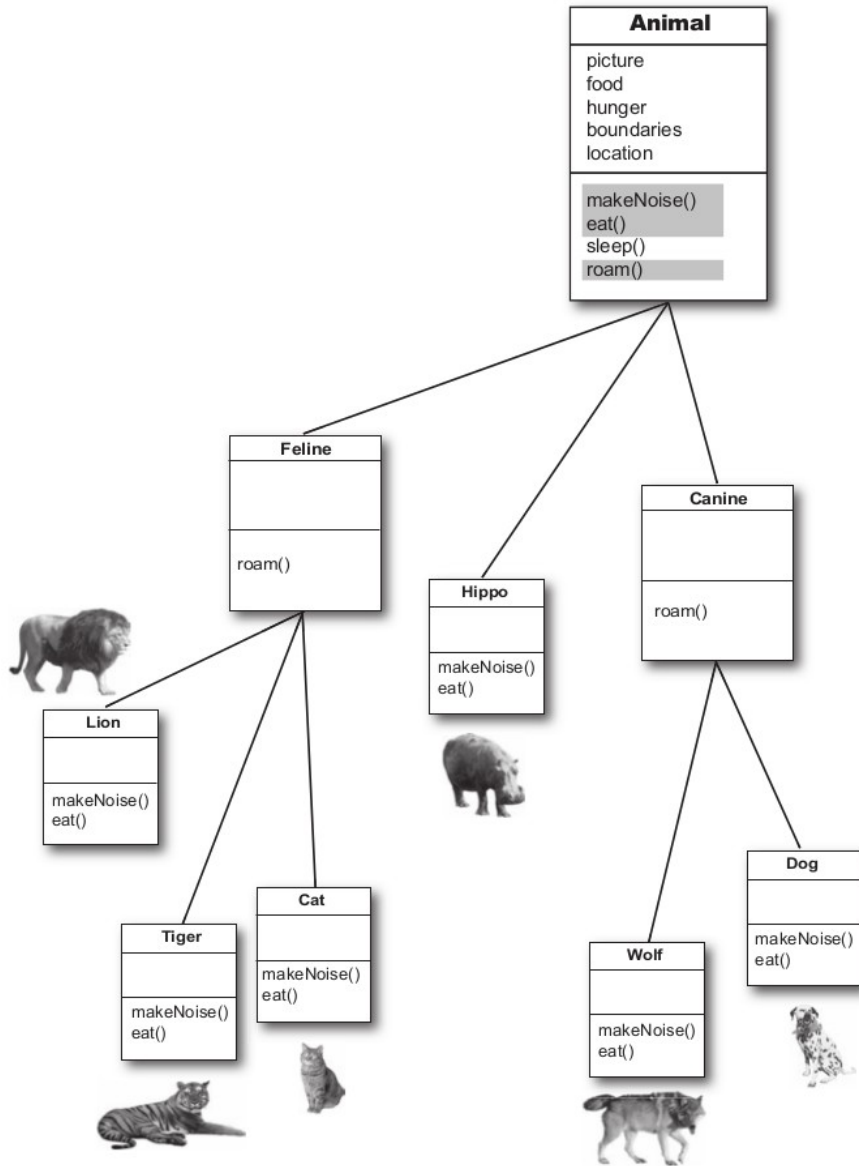
A

```
boolean scare(int x) {
    System.out.println("a bite?");
    return true;
}
```

B

Serious Polymorphism





We know we can say:

```
Wolf aWolf = new Wolf();
```

A Wolf reference to a
Wolf object.



Wolf



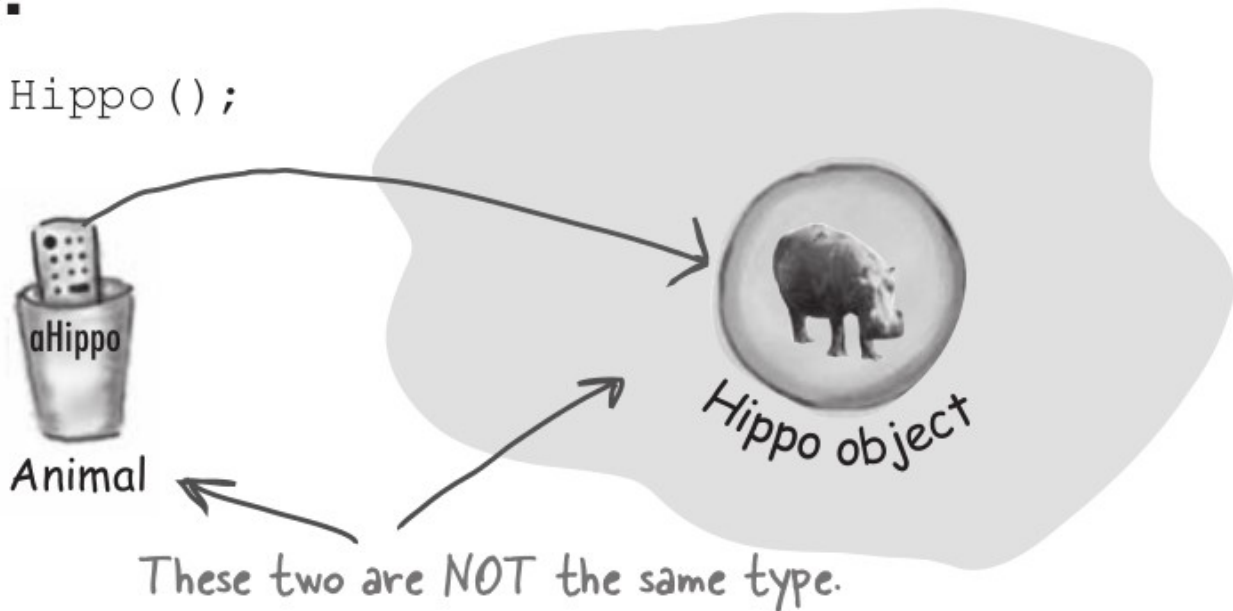
Wolf object

These two are the same type.

And we know we can say:

```
Animal aHippo = new Hippo();
```

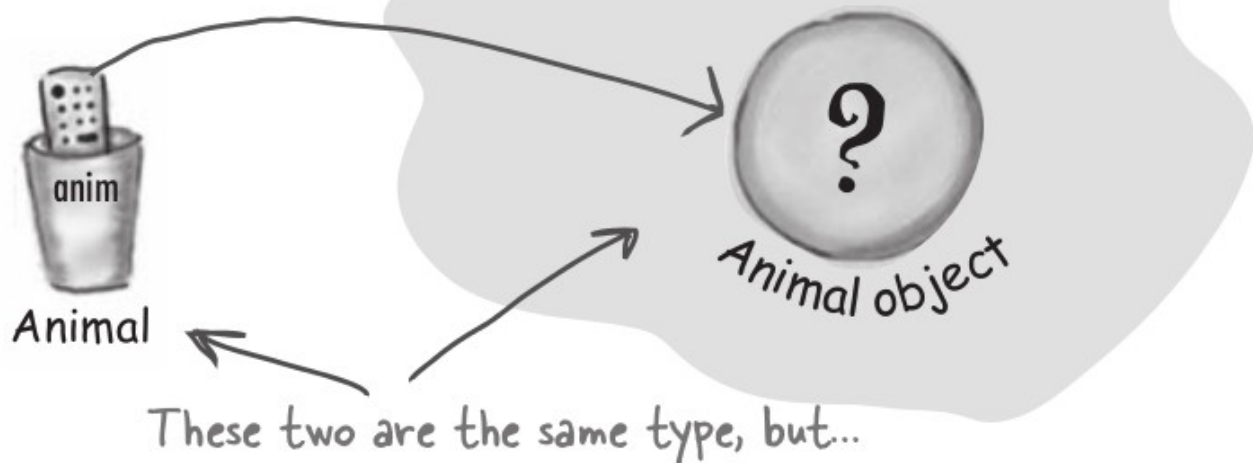
Animal reference to
a Hippo object.



But here's where it gets weird:

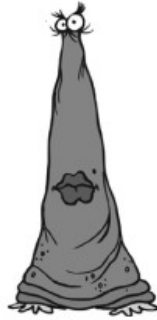
```
Animal anim = new Animal();
```

Animal reference to
an Animal object.



These two are the same type, but...
what the heck does an Animal object look like?

**What does a new `Animal()` object
*look like?***



scary objects



```
abstract public class Canine extends Animal
{
    public void roam() { }
}
```

```
public class MakeCanine {
```

```
    public void go() {
```

```
        Canine c;
```

```
        c = new Dog();
```

```
        c = new Canine();
```

```
        c.roam();
```

```
    }
```

```
}
```

This is OK, because you can always assign a subclass object to a superclass reference, even if the superclass is abstract.

class Canine is marked abstract, so the compiler will NOT let you do this.

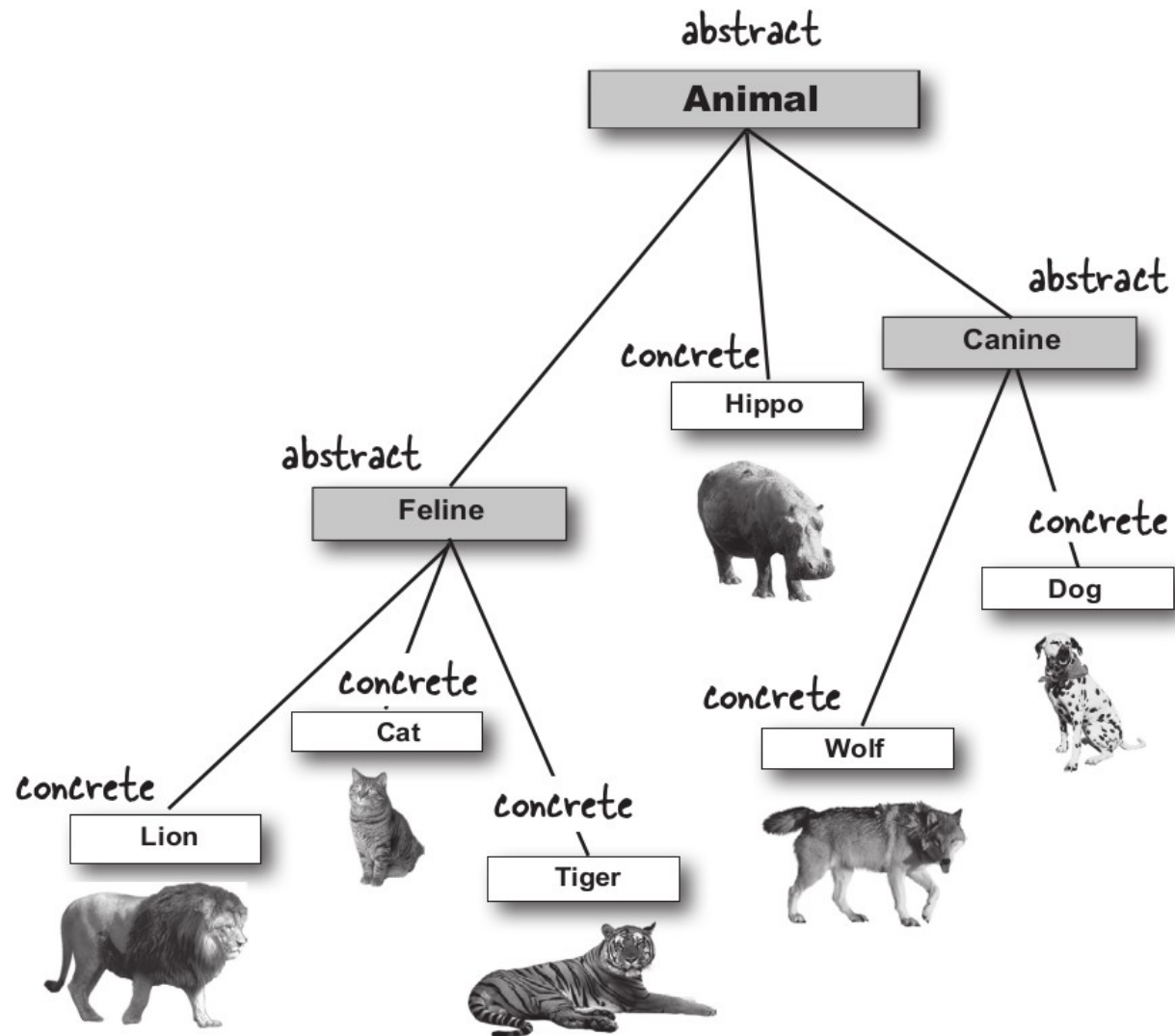
File Edit Window Help BeamMeUp

```
% javac MakeCanine.java
```

```
MakeCanine.java:5: Canine is abstract;
cannot be instantiated
```

```
        c = new Canine();
                ^
```

```
1 error
```



An abstract method has no body!

```
public abstract void eat();
```

No method body!
End it with a semicolon.



If you declare an abstract *method*, you **MUST** mark the *class* abstract as well. You can't have an abstract method in a non-abstract class.



You **MUST** implement all abstract methods

I have wonderful news, mother. Joe finally implemented all his abstract methods! Now everything is working just the way we planned...

Implementing an abstract method is just like overriding a method.



Polymorphism in action

version
1

MyDogList
Dog[] dogs int nextIndex
add(Dog d)

```
public class MyDogList {
```

```
    private Dog [] dogs = new Dog[5];
```

Use a plain old Dog array behind the scenes.

```
    private int nextIndex = 0;
```

We'll increment this each time a new Dog is added.

```
    public void add(Dog d) {
```

```
        if (nextIndex < dogs.length) {
```

```
            dogs[nextIndex] = d;
```

If we're not already at the limit of the dogs array, add the Dog and print a message.

```
            System.out.println("Dog added at " + nextIndex);
```

```
            nextIndex++;
```

increment, to give us the next index to use

```
        }
```

```
    }
```

```
}
```

Building our own Animal-specific list

version
2

MyAnimalList

Animal[] animals
int nextIndex

add(Animal a)

```
public class MyAnimalList {
```

```
    private Animal[] animals = new Animal[5];  
    private int nextIndex = 0;
```

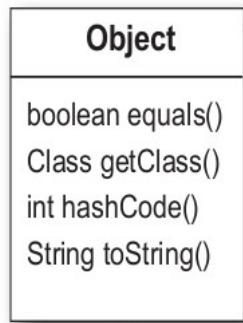
```
    public void add(Animal a) {  
        if (nextIndex < animals.length) {  
            animals[nextIndex] = a;  
            System.out.println("Animal added at " + nextIndex);  
            nextIndex++;  
        }  
    }  
}
```

Don't panic. We're not making a new Animal object; we're making a new array object, of type Animal. (Remember, you cannot make a new instance of an abstract type, but you CAN make an array object declared to HOLD that type.)

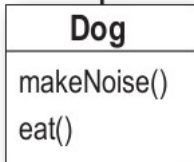
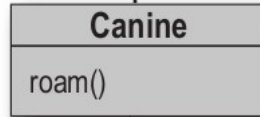
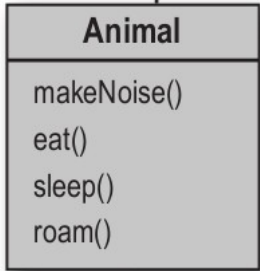
```
public class AnimalTestDrive{  
    public static void main (String[] args) {  
        MyAnimalList list = new MyAnimalList();  
        Dog a = new Dog();  
        Cat c = new Cat();  
        list.add(a);  
        list.add(c);  
    }  
}
```

File Edit Window Help Harm

```
% java AnimalTestDrive  
  
Animal added at 0  
  
Animal added at 1
```



Animal doesn't explicitly extend anything, so it's automatically a subclass of Object.



① equals(Object o)

```
Dog a = new Dog();
Cat c = new Cat();

if (a.equals(c)) {
    System.out.println("true");
} else {
    System.out.println("false");
}
```

```
File Edit Window Help Stop

% java TestObject

false
```

Tells you if two objects are considered 'equal' (we'll talk about what 'equal' really means in appendix B).

③ hashCode()

```
Cat c = new Cat();
System.out.println(c.hashCode());
```

```
File Edit Window Help Drop

% java TestObject

8202111
```

Prints out a hashCode for the object (for now, think of it as a unique ID).

② getClass()

```
Cat c = new Cat();
System.out.println(c.getClass());
```

```
File Edit Window Help Faint

% java TestObject

class Cat
```

Gives you back the class that object was instantiated from.

④ toString()

```
Cat c = new Cat();
System.out.println(c.toString());
```

```
File Edit Window Help LapseIntoComa

% java TestObject

Cat@7d277f
```

Prints out a String message with the name of the class and some other number we rarely care about.

```
ArrayList<Dog> myDogArrayList = new ArrayList<Dog>();  
Dog aDog = new Dog();  
myDogArrayList.add(aDog);  
Dog d = myDogArrayList.get(0);
```

← Make an ArrayList declared to hold Dog objects.
← Make a Dog.
← Add the Dog to the list.
← Assign the Dog from the list to a new Dog reference variable. (Think of it as though the get() method declares a Dog return type because you used ArrayList<Dog>.)

But what happens when you declare it as ArrayList<Object>? If you want to make an ArrayList that will literally take *any* kind of Object, you declare it like this:

```
ArrayList<Object> myDogArrayList = new ArrayList<Object>();  
Dog aDog = new Dog();  
myDogArrayList.add(aDog);
```

← Make an ArrayList declared to hold any type of Object.
← Make a Dog.
← Add the Dog to the list. (These two steps are the same.)

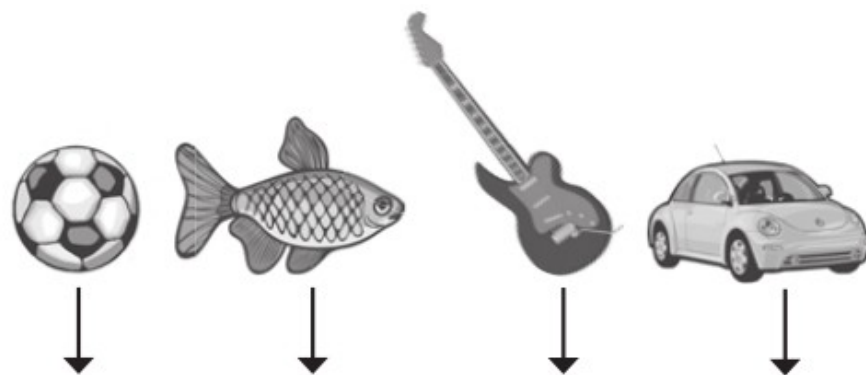
But what happens when you try to get the Dog object and assign it to a Dog reference?

```
Dog d = myDogArrayList.get(0);
```

 NO!! Won't compile!! When you use ArrayList<Object>, the get() method returns type Object. The Compiler knows only that the object inherits from Object (somewhere in its inheritance tree) but it doesn't know it's a Dog!!

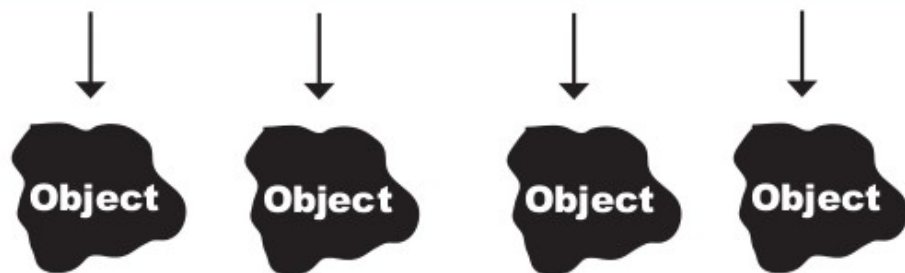
*Everything comes out of an ArrayList<Object> as a reference of type **Object**, regardless of what the actual object is, or what the reference type was when you added the object to the list.*

The objects go IN
as **SoccerBall**,
Fish, **Guitar**, and
Car.



ArrayList<Object>

But they come
OUT as though
they were of type
Object.



Objects come out of
an **ArrayList<Object>**
acting like they're
generic instances
of class **Object**. The
Compiler cannot
assume the object
that comes out is of
any type other than
Object.

BAD



```
public void go() {  
    Dog aDog = new Dog();  
    Dog sameDog = getObject(aDog);  
}
```

```
public Object getObject(Object o) {  
    return o;  
}
```

We're returning a reference to the same Dog, but as a return type of Object. This part is perfectly legal. Note: this is similar to how the get() method works when you have an ArrayList<Object> rather than an ArrayList<Dog>.

This line won't work! Even though the method returned a reference to the very same Dog the argument referred to, the return type Object means the compiler won't let you assign the returned reference to anything but Object.

I don't know what you're talking about. Sit? Stay? bark? Hmmmm...I don't recall knowing those.



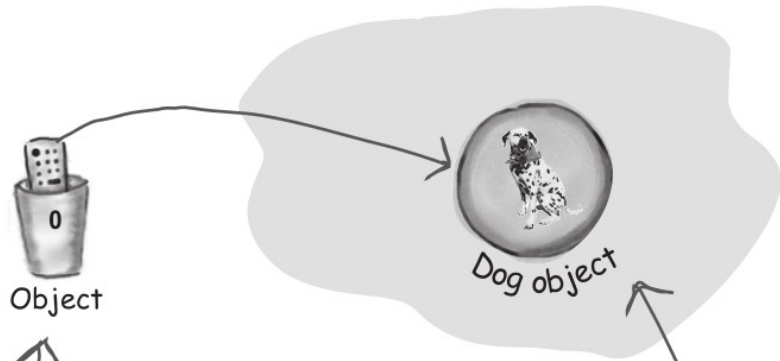
File Edit Window Help Remember

```
DogPolyTest.java:10: incompatible types  
found   : java.lang.Object  
required: Dog  
        Dog sameDog = getObject(aDog);  
1 error
```

GOOD
😊

```
public void go() {  
    Dog aDog = new Dog();  
    Object sameDog = getObject(aDog);  
}  
  
public Object getObject(Object o) {  
    return o;  
}
```

← This works (although it may not be very useful, as you'll see in a moment) because you can assign **ANYTHING** to a reference of type `Object`, since every class passes the IS-A test for `Object`. Every object in Java is an instance of type `Object`, because every class in Java has `Object` at the top of its inheritance tree.



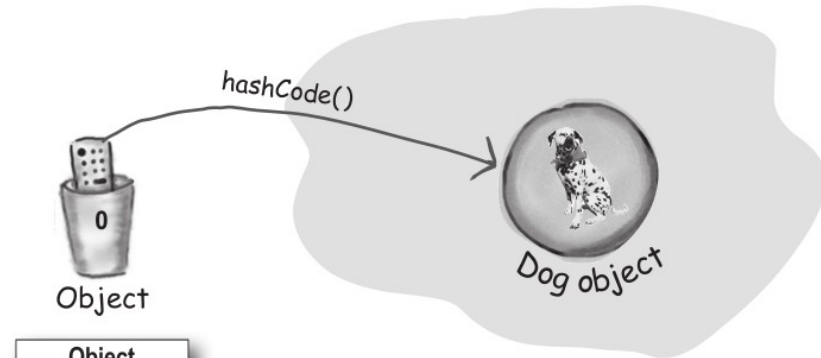
When you get an object reference from an `ArrayList<Object>` (or any method that declares `Object` as the return type), it comes back as a polymorphic reference type of `Object`. So you have an `Object` reference to (in this case) a Dog instance.

The compiler decides whether you can call a method based on the reference type, not the actual *object* type.

```
Object o = al.get(index);
int i = o.hashCode();
```

← This is fine. Class `Object` has a `hashCode()` method, so you can call that method on ANY object in Java.

Won't compile! → `o.bark();` ← Can't do this!! The `Object` class has no idea what it means to `bark()`. Even though YOU know it's really a `Dog` at that index, the compiler doesn't.



Object
<code>equals()</code>
<code>getClass()</code>
<code>hashCode()</code>
<code>toString()</code>

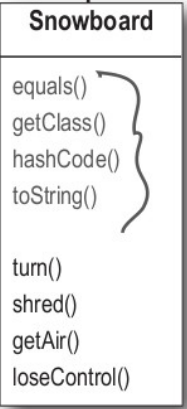
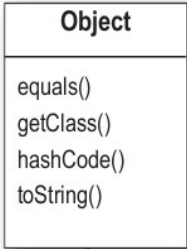
The method you're calling on a reference **MUST** be in the class of that reference type. Doesn't matter what the actual object is.

`o.hashCode();`

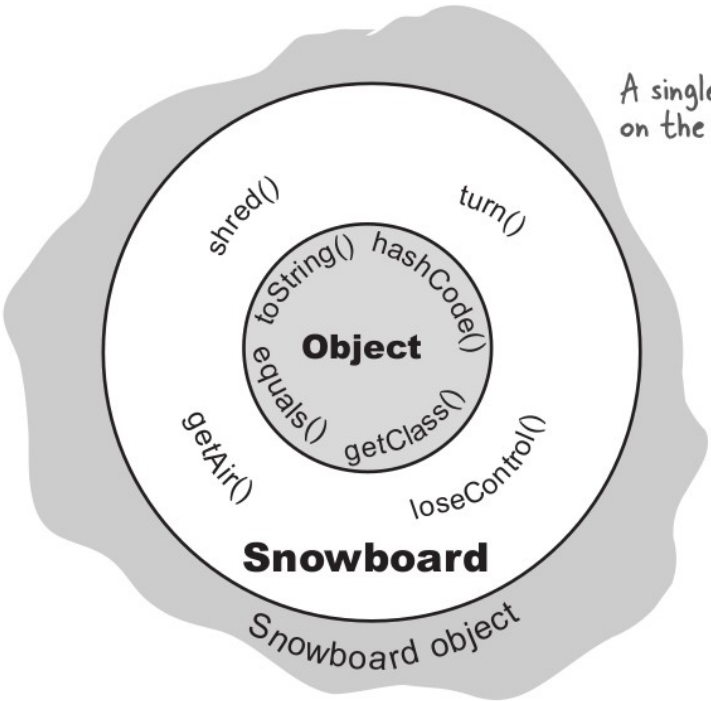
The "o" reference was declared as type `Object`, so you can call methods only if those methods are in class `Object`.



He treats me like an Object. But I can do so much more...if only he'd see me for what I *really* am.



Snowboard inherits methods from superclass Object and adds four more.

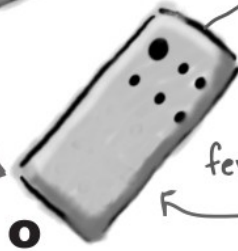
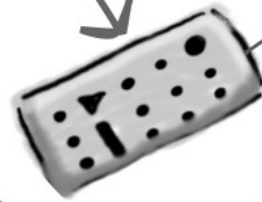


A single object on the heap.

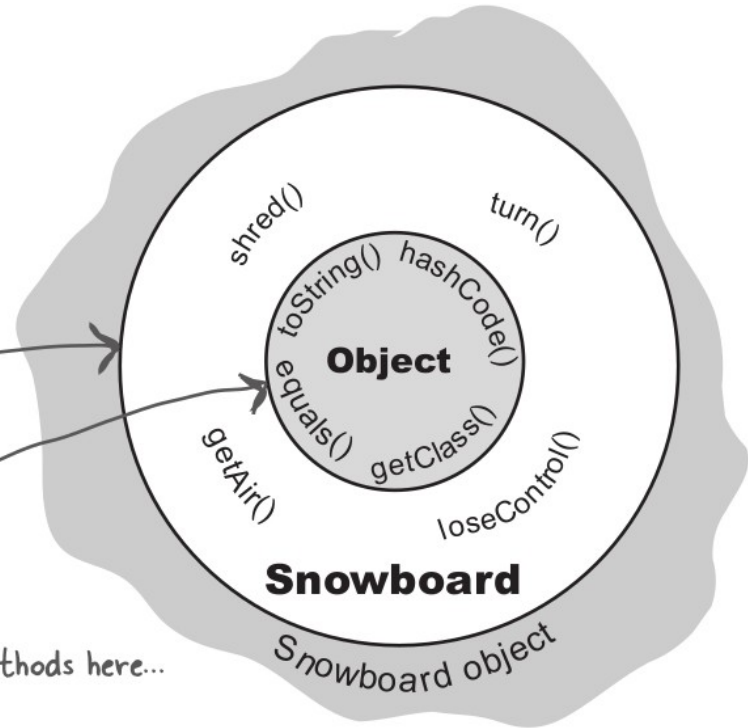
There is only *ONE* object on the heap here. A Snowboard object. But it contains both the Snowboard class parts of itself and the Object class parts of itself.

```
Snowboard s = new Snowboard();  
Object o = s;
```

The Snowboard remote control (reference) has more buttons than an Object remote control. The Snowboard remote can see the full Snowboardness of the Snowboard object. It can access all the methods in Snowboard, including both the inherited Object methods and the methods from class Snowboard.



fewer methods here...



The Object reference can see only the Object parts of the Snowboard object. It can access only the methods of class Object. It has fewer buttons than the Snowboard remote control.

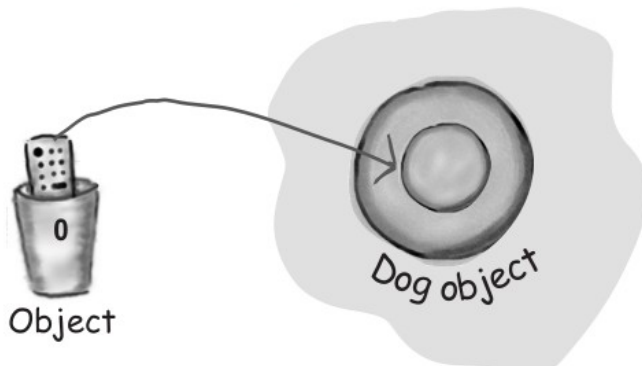


Wait a minute... what good is a Dog if it comes out of an `ArrayList<Object>` and it can't do any Dog things? There's gotta be a way to get the Dog back to a state of Dogness...

I hope it doesn't hurt.
And what's so wrong with staying an Object? OK, I can't fetch, sure, but I can give you a real nice hashCode.

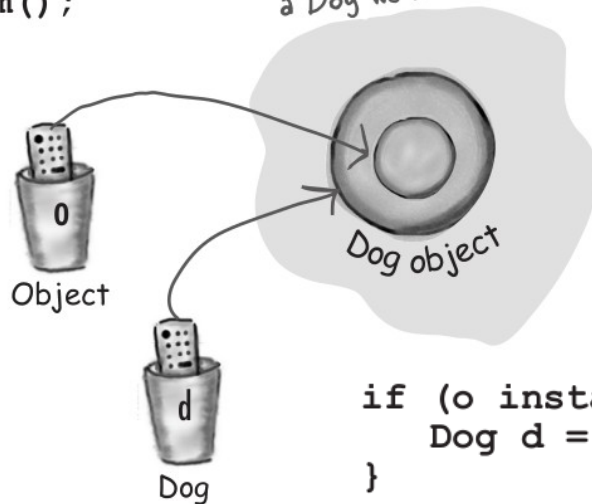


Casting an object reference back to its *real* type.



```
Object o = al.get(index);
```

```
Dog d = (Dog) o; ← cast the Object back to  
d.roam();           a Dog we know is there.
```



```
if (o instanceof Dog) {  
    Dog d = (Dog) o;  
}
```

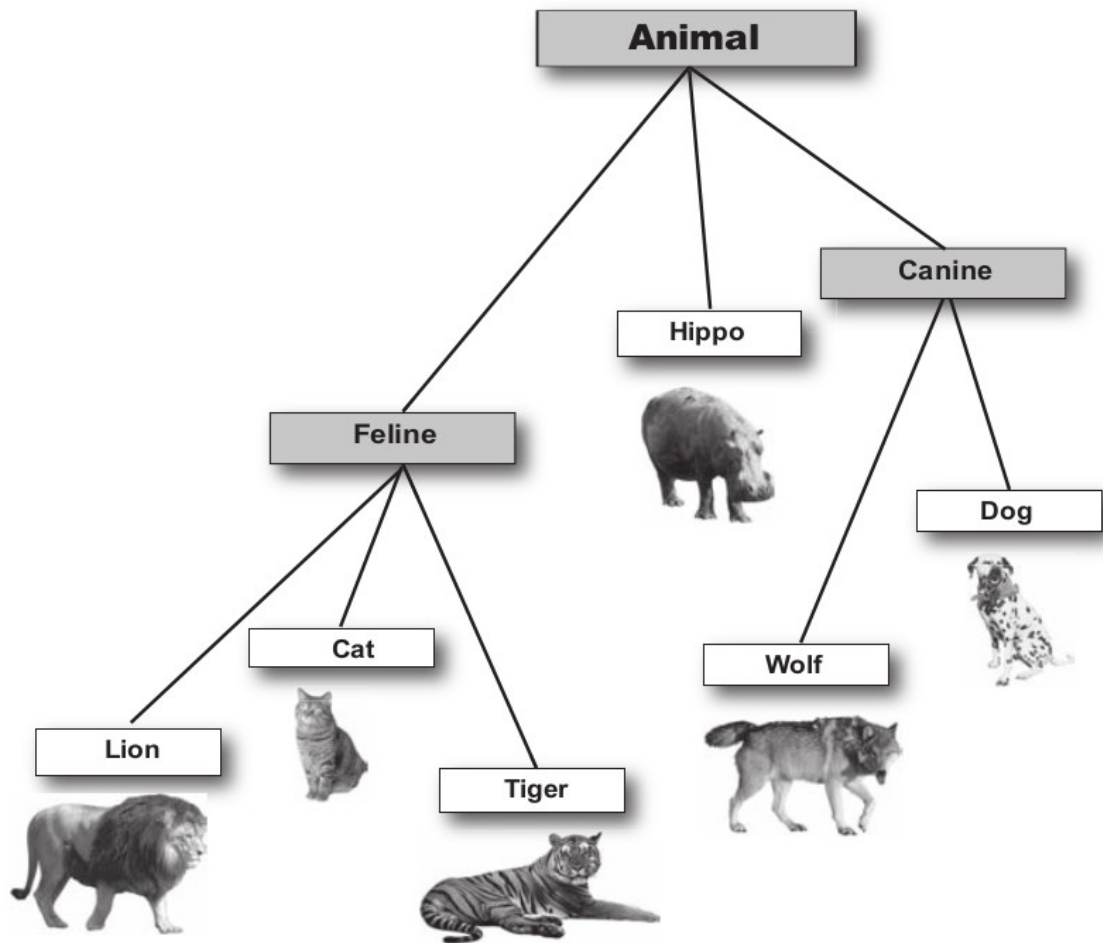
Think about what **YOU** would do if **YOU** were the Dog class programmer and needed to modify the Dog so that it could do Pet things, too. We know that simply adding new Pet behaviors (methods) to the Dog class will work, and won't break anyone else's code.

But... this is a PetShop program. It has more than just Dogs! And what if someone wants to use your Dog class for a program that has *wild* Dogs? What do you think your options might be, and without worrying about how Java handles things, just try to imagine how you'd *like* to solve the problem of modifying some of your Animal classes to include Pet behaviors.

① Option one

We take the easy path, and put pet methods in class `Animal`.

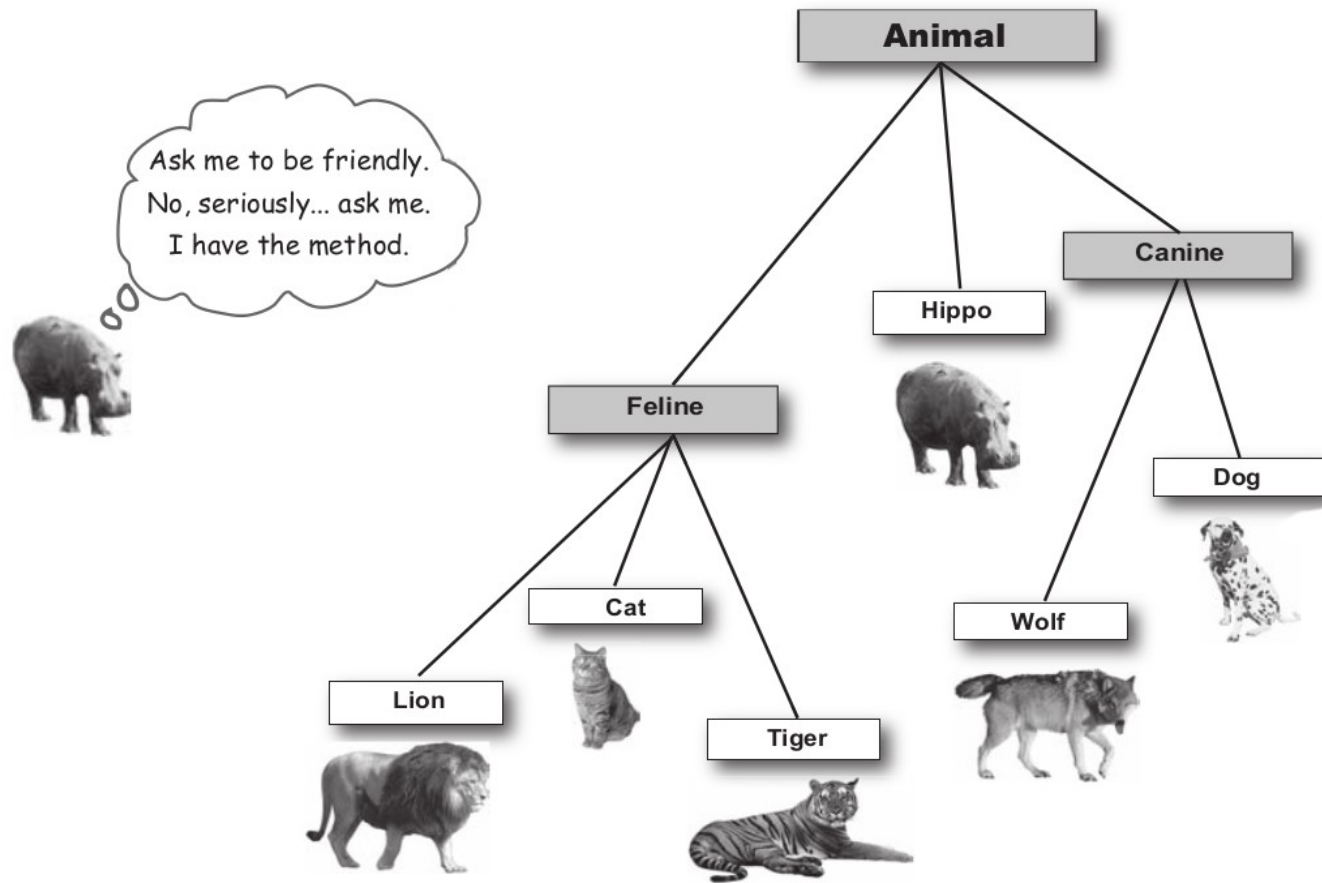
put all the pet
method code up here
for inheritance.



② Option two

We start with Option One, putting the pet methods in class Animal, but we make the methods abstract, forcing the Animal subclasses to override them.

put all the pet methods up here, but with no implementations. Make all pet methods abstract

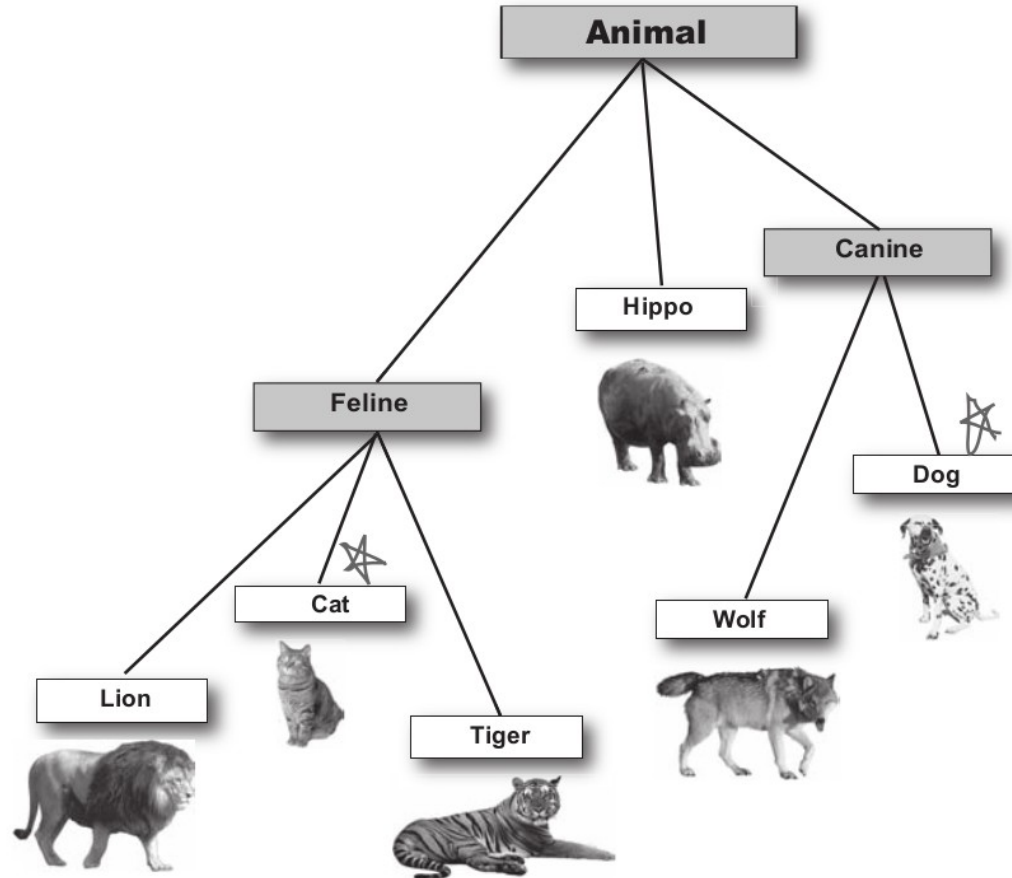


③

Option three

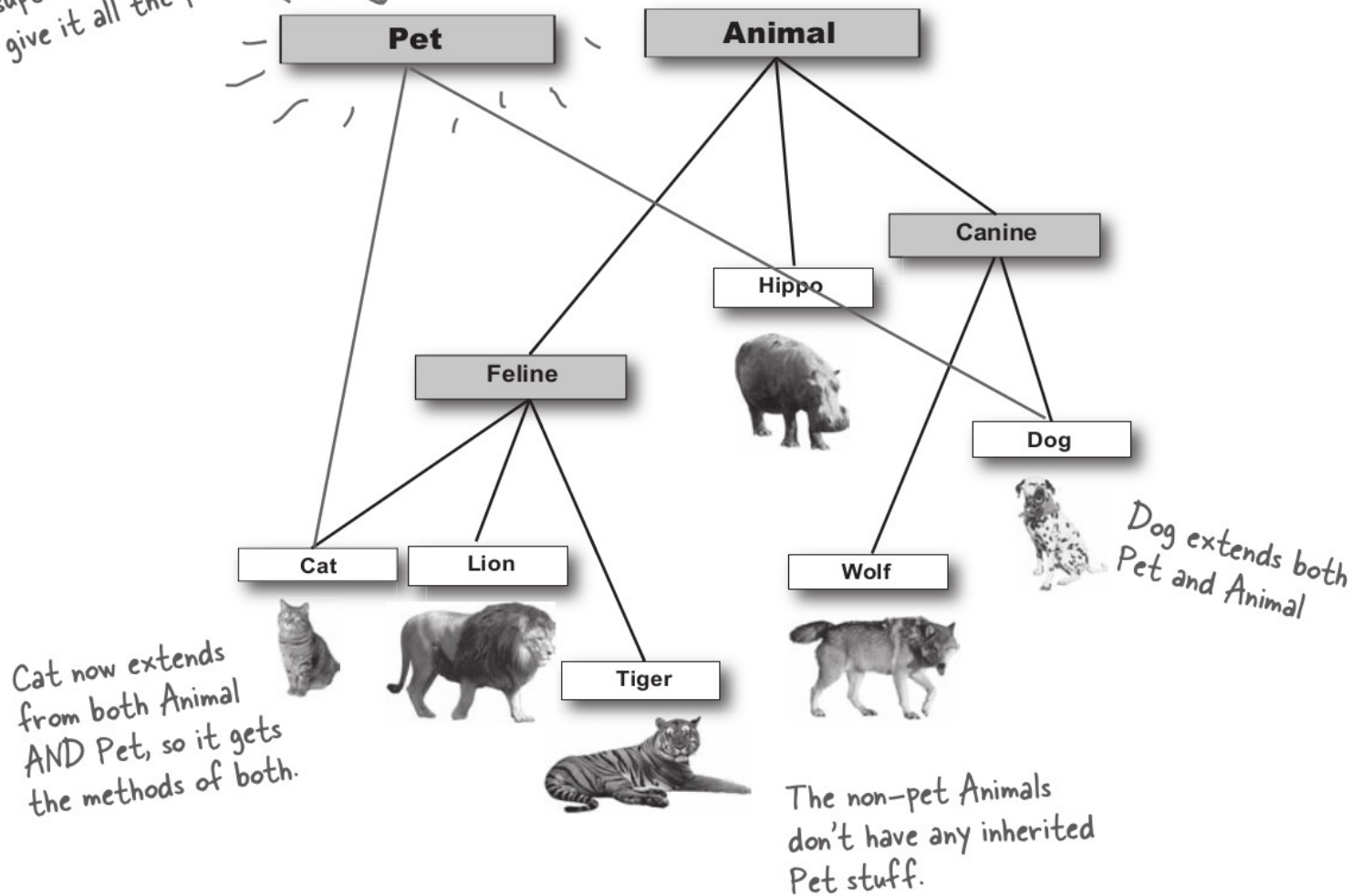
Put the pet methods **ONLY** in the classes where they belong.

★ Put the pet methods **ONLY** in the Animal classes that can be pets, instead of in Animal.



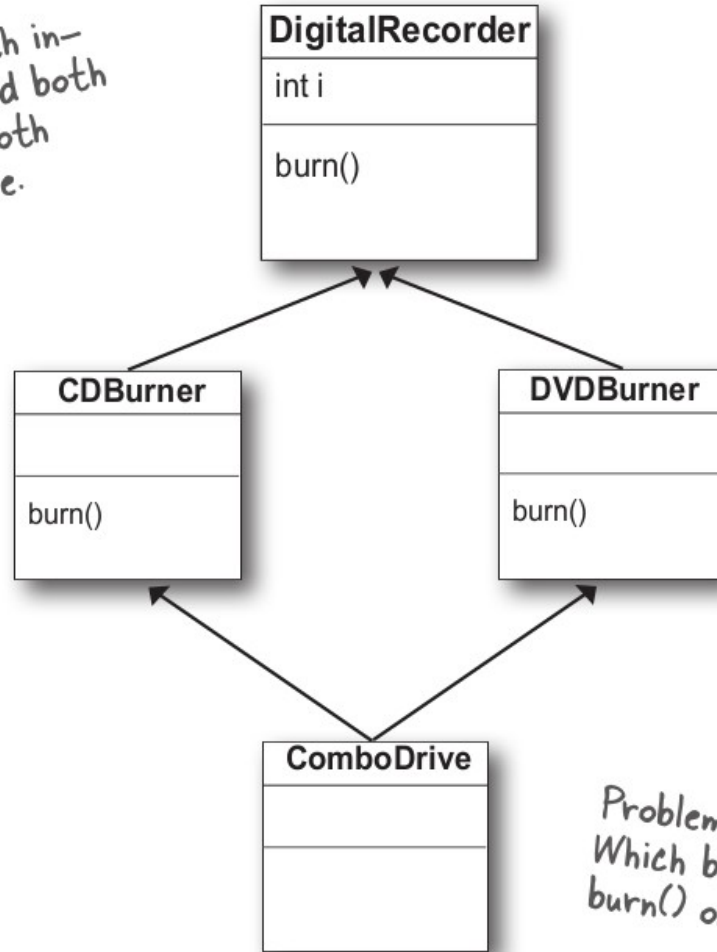
We make a new abstract superclass called Pet, and give it all the pet methods.

It looks like we need **TWO** superclasses at the top.



Deadly Diamond of Death

CDBurner and DVDBurner both inherit from DigitalRecorder, and both override the burn() method. Both inherit the "i" instance variable.



Imagine that the "i" instance variable is used by both CDBurner and DVDBurner, with different values. What happens if ComboDrive needs to use both values of "i"?

Problem with multiple inheritance. Which burn() method runs when you call burn() on the ComboDrive?

<i>Pet</i>
<i>abstract void beFriendly();</i> <i>abstract void play();</i>

A Java interface is like a 100% pure abstract class.

All methods in an interface are abstract, so any class that IS-A Pet **MUST** implement (i.e. override) the methods of Pet.

To DEFINE an interface:

```
public interface Pet {...}
```

Use the keyword "interface" instead of "class"

To IMPLEMENT an interface:

```
public class Dog extends Canine implements Pet {...}
```

Use the keyword "implements" followed by the interface name. Note that when you implement an interface you still get to extend a class

Making and implementing the Pet interface

You say "interface" instead of "class" here.

```
public interface Pet {  
    public abstract void beFriendly();  
    public abstract void play();  
}
```

Interface methods are implicitly public and abstract, so typing in "public" and "abstract" is optional (in fact, it's not considered "good style" to type the words in, but we did here just to reinforce it).

All interface methods are abstract, so they **MUST** end in semicolons. Remember, they have no body!

Dog IS-A Animal
and Dog IS-A Pet

```
public class Dog extends Canine implements Pet {
```

You say "implements" followed by the name of the interface.

```
    public void beFriendly() {...}
```

```
    public void play() {...}
```

```
    public void roam() {...}
```

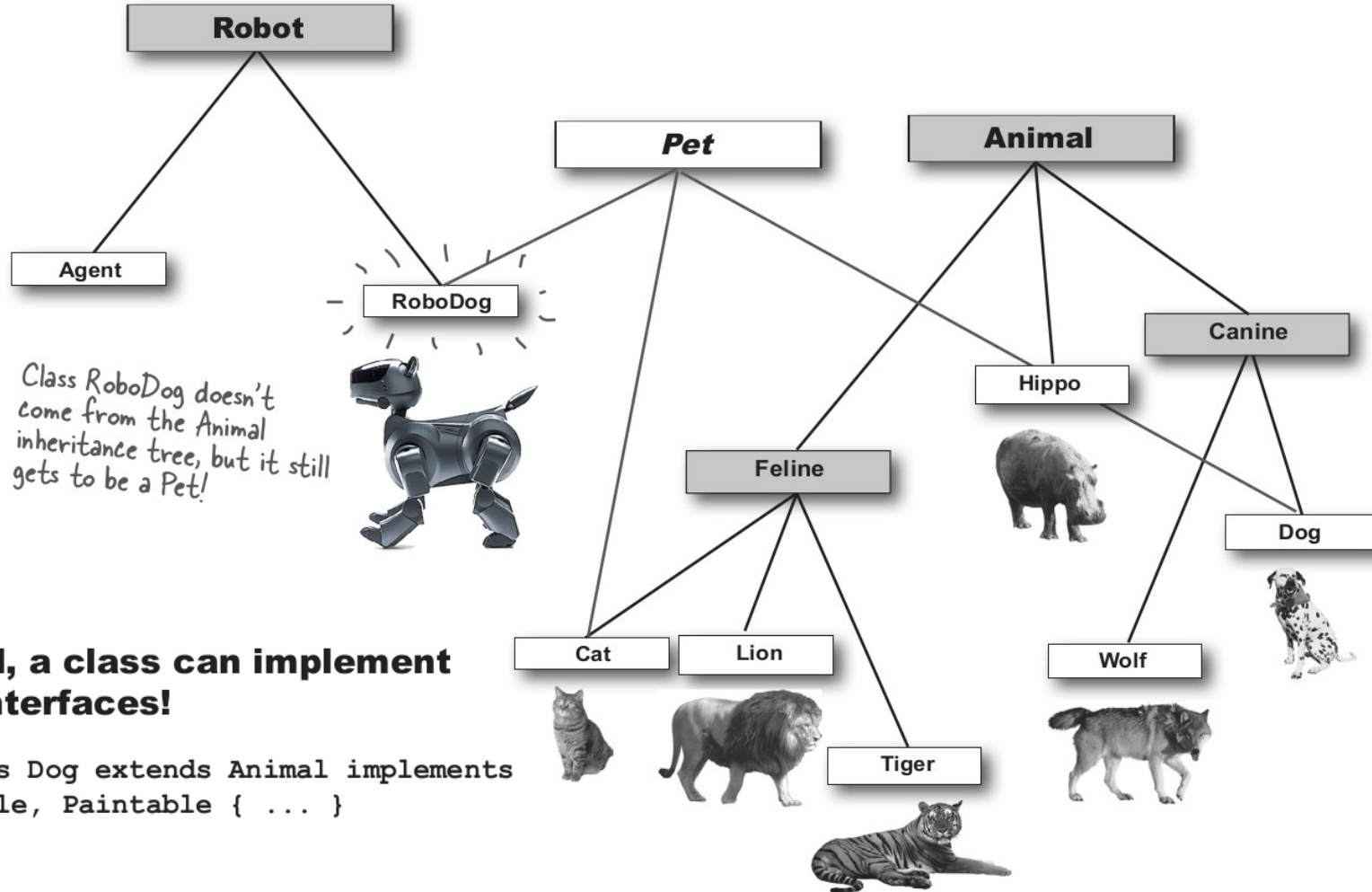
```
    public void eat() {...}
```

```
}
```

You SAID you are a Pet, so you **MUST** implement the Pet methods. It's your contract. Notice the curly braces instead of semicolons.

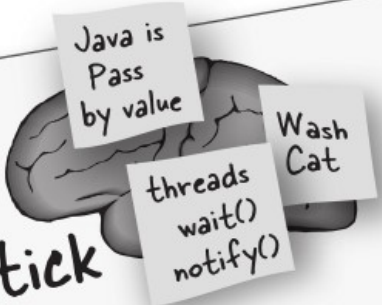
These are just normal overriding methods.

Classes from *different* inheritance trees can implement the *same* interface.



Better still, a class can implement *multiple* interfaces!

```
public class Dog extends Animal implements
    Pet, Saveable, Paintable { ... }
```



Make it Stick

Roses are red, violets are blue.

Extend only **one**, but **implement two**.

Java weighs in on family values:

Single Parents Only!! A Java class can have only **one** parent (superclass), and that parent class defines who you are. But you can implement multiple interfaces, and those interfaces define *roles you can play*.

How do you know whether to make a class, a subclass, an *abstract* class, or an interface?

- Make a class that doesn't extend anything (other than `Object`) when your new class doesn't pass the IS-A test for any other type.
- Make a subclass (in other words, *extend* a class) only when you need to make a *more specific* version of a class and need to override or add new behaviors.
- Use an abstract class when you want to define a *template* for a group of subclasses, and you have at least *some* implementation code that all subclasses could use. Make the class abstract when you want to guarantee that nobody can make objects of that type.
- Use an interface when you want to define a *role* that other classes can play, regardless of where those classes are in the inheritance tree.

Invoking the superclass version of a method

```
abstract class Report {  
    void runReport() {  
        // set-up report  
    }  
    void printReport() {  
        // generic printing  
    }  
}
```

← superclass version of the method does important stuff that subclasses could use

```
class BuzzwordsReport extends Report {  
  
    void runReport() {  
        super.runReport();  
        buzzwordCompliance();  
        printReport();  
    }  
    void buzzwordCompliance() {...}  
}
```

← call superclass version, then come back and do some subclass-specific stuff

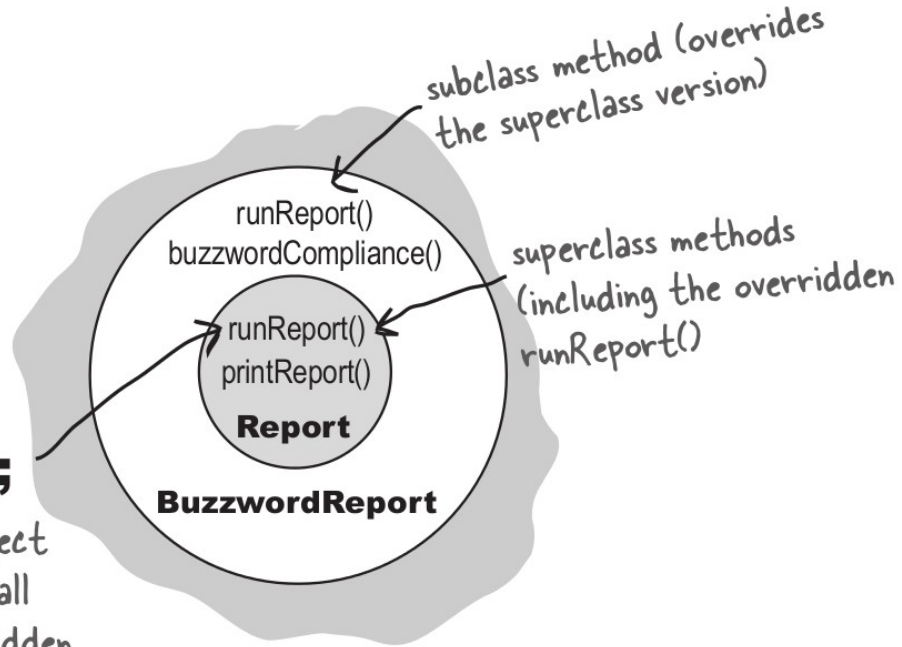
If method code inside a
BuzzwordReport subclass says:

super.runReport();

the runReport() method inside
the superclass Report will run

super.runReport();

A reference to the subclass object
(BuzzwordReport) will always call
the subclass version of an overridden
method. That's polymorphism.
But the subclass code can call
super.runReport() to invoke the
superclass version.



The super keyword is really a reference
to the superclass portion of an object.
When subclass code uses super, as in
super.runReport(), the superclass version of
the method will run.